

Commander.js



build passing

[Install Size](#)

The complete solution for [node.js](#) command-line interfaces.

Read this in other languages: English | [简体中文](#)

- [Commander.js](#)
- [Installation](#)
- [Quick Start](#)
- [Declaring program variable](#)
- [Options](#)
- [Common option types, boolean and value](#)
- [Default option value](#)

1

2

3

4

8

7

9

5

- [Additional documentation](#)
 - [Support](#)
 - [Commander for enterprise](#)
 - For information about terms used in this document see: [terminology](#)
- ### Installation
- npm install commander

- [Custom event listeners](#)
- [Bits and pieces](#)
- [.parse\(\) and .parseAsync\(\)](#)
- [Parsing Configuration](#)
- [Legacy options as properties](#)
- [TypeScript](#)
- [createCommand\(\)](#)
- [Node options such as --harmony](#)
- [Debugging stand-alone executables](#)
- [subcommands](#)
- [Display error](#)
- [Override exit and output handling](#)

- [Stand-alone executable \(sub\)commands](#)
- [Life cycle hooks](#)
- [Automated help](#)
- [Custom help](#)
- [Display help after errors](#)
- [Display help from code](#)
- [.name](#)
- [.usage](#)
- [.description and .summary](#)
- [.helpOption\(flags, description\)](#)
- [.addHelpCommand\(\)](#)
- [More configuration](#)

- [Other option types, negatable boolean and boolean|value](#)
- [Required option](#)
- [Variadic option](#)
- [Version option](#)
- [More configuration](#)
- [Custom option processing](#)
- [Commands](#)
- [Command-arguments](#)
- [More configuration](#)
- [Custom argument processing](#)
- [Action handler](#)

Quick Start

You write code to describe your command line interface. Commander looks after parsing the arguments into options and command-arguments, displays usage errors for problems, and implements a help system.

Commander is strict and displays an error for unrecognised options. The two most used option types are a boolean option, and an option which takes its value from the following argument.

Example file: [split.js](#)

```
const { program } =
  require('commander');

program
  .option('--first')
  .option('-s, --separator <char>');

program.parse();

const options = program.opts();
```

```
const limit = options.first ? 1 :
  undefined;
console.log(program.args[0].split(options.separator,
  limit));
```

```
$ node split.js -s / --fits a/b/c
error: unknown option '--fits'
(Did you mean --first?)
$ node split.js -s / --first a/b/c
[ 'a' ]
```

Here is a more complete program using a subcommand and with descriptions for the

help. In a multi-command program, you have an action handler for each command (or stand-alone executables for the commands).

Example file: [string-util.js](#)

```
const { Command } =
  require('commander');
const program = new Command();

program
  .name('string-util')
  .description('CLI to some JavaScript string utilities')
```

9

10

11

12

```
-s, --separator <char> separator
-h, --help          display help
for command

$ node string-util.js split --
separator=/ a/b/c
[ 'a', 'b', 'c' ]

More samples can be found in the examples directory.
```

```
<string>
Split a string into substrings and
display as an array.

Arguments:
  string      string to
  split       split
Options:
  --first    the first substring
display just
```

```
.option('-s, --separator <char>',
  'separator character', '')
.action((str, options) => {
  const limit = options.first ? 1 :
    undefined;
  console.log(str.split(options.separator,
    limit));
});

$ node string-util.js help split
Usage: string-util split [options]
```

```
.version('0.8.0');

program.command('split')
  .description('Split a string into
  substrings and display as an
  array')
  .argument('<string>', 'string to
  split')
  .option('--first',
    'display just the first
    substring')
```

16

15

14

13

Declaring program variable Commander exports a global object which is convenient for quick programs. This is used in the examples in this README for brevity. <pre>// CommonJS (.cjs) const { program } = require('commander');</pre> For larger programs which may use commander in multiple ways, including unit 17	testing, it is better to create a local Command object to use. <pre>// CommonJS (.cjs) const { Command } = require('commander'); const program = new Command(); // ECMAScript (.mjs) import { Command } from 'commander'; const program = new Command();</pre> 18	<pre>// TypeScript (.ts) import { Command } from 'commander'; const program = new Command();</pre> Options Options are defined with the .option() method, also serving as documentation for the options. Each option can have a short flag (single character) and a long name, separated by a comma or space or vertical bar (' '). 19	The parsed options can be accessed by calling .opts() on a Command object, and are passed to the action handler. Multi-word options such as “–template-engine” are camel-cased, becoming program.opts().templateEngine etc. An option and its option-argument can be separated by a space, or combined into the same argument. The option-argument can follow the short option directly or follow an = for a long option. 20
<pre>const options = program.opts(); program.parse(process.argv); 'flavour of pizza'; .option('-p, --pizza-type <type>', size() .option('-s, --small', 'small pizza 'output extra debugging') .option('-d, --debug', program</pre> 24	The two most used option types are a boolean option, and an option which takes its value from the following argument (declared with angle brackets like <value>). Both are undefined unless specified on command line. Example file: options-common.js 23 Common option types, boolean and value	There are additional related routines for when .opts() is not enough: .optWithGlobals() returns merged local and global option values .getOptionValue() and .setOptionValue() work with a single option value .getOptValueSource() and .setOptValueWithSource() include where the option value came from 22	You can use -- to indicate the end of the options, and any remaining arguments will be used without being interpreted. By default options on the command line are not positional, and can be specified before or after other arguments. <pre>serve -p 80 serve -p80 serve --port 80 serve --port=80</pre> 21
<pre>if (options.debug) console.log(options); console.log('pizza details:'); if (options.small) console.log('- small pizza size'); if (options.pizzaType) console.log(`-   \${options.pizzaType}`); \$ pizza-options -p error: option '-p, --pizza-type <type>' argument missing \$ pizza-options -d -s -p vegetarian { debug: true, small: true, pizzaType:</pre> 25	<pre>'vegetarian' } pizza details: - small pizza size - vegetarian \$ pizza-options --pizza-type=cheese pizza details: - cheese Multiple boolean short options may be combined together following the dash, and may be followed by a single short option taking a value. For example -d -s -p cheese</pre> 26	may be written as -ds -p cheese or even -dsp cheese. Options with an expected option-argument are greedy and will consume the following argument whatever the value. So --id -xyz reads -xyz as the option-argument. program.parse(arguments) processes the arguments, leaving any args not consumed by the program options in the program.args array. The parameter is optional and defaults to process.argv. 27	Default option value You can specify a default value for an option. Example file: options-defaults.js <pre>program .option('-c, --cheese <type>', 'add the specified type of cheese', 'blue'); program.parse();</pre> 28
<pre>\$ pizza-options --sauce mozzarella cheese You ordered a pizza with sauce and \$ pizza-options); console.log('You ordered a pizza with {options.cheese} cheese'; false) ? 'no cheese' : '\$ const cheeseStr = (options.cheese === 'sauce' : 'no sauce'; const sauceStr = options.sauce ?</pre> 32	<pre>program .option('--no-sauce', 'Remove sauce') .option('--cheese <flavour>', 'cheese flavour', 'mozzarella') .option('--no-cheese', 'plain with no cheese') .parse(); const options = program.opts();</pre> 31	You can define a boolean option long name with a leading no - to set the option value to false when used. Defined alone this also makes the option true by default. If you define --foo first, adding --no-foo does not change the default value from what it would otherwise be. Example file: options-negatable.js 30 Other option types, negatable boolean and boolean value	<pre>console.log('cheese: \$ {program.opts().cheese}'); \$ pizza-options cheese: blue \$ pizza-options --cheese stillon cheese: stillon</pre> 29

<pre>error: unknown option '--sauce' \$ pizza-options --cheese=blue You ordered a pizza with sauce and blue cheese \$ pizza-options --no-sauce --no-cheese You ordered a pizza with no sauce and no cheese You can specify an option which may be used as a boolean option but may optionally take an option-argument (declared with square brackets like --optional [value]). Example file: options-boolean-or-value.js</pre>	<pre>program .option('-c, --cheese [type]', 'Add cheese with optional type'); program.parse(process.argv); const options = program.opts(); if (options.cheese === undefined) console.log('no cheese');</pre>	<pre>else if (options.cheese === true) console.log('add cheese'); else console.log(`add cheese type \$ {options.cheese}`); \$ pizza-options no cheese \$ pizza-options --cheese add cheese \$ pizza-options --cheese mozzarella add cheese type mozzarella</pre>	Options with an optional option-argument are not greedy and will ignore arguments starting with a dash. So id behaves as a boolean option for --id -5, but you can use a combined form if needed like --id=-5. For information about possible ambiguous cases, see options taking varying arguments. Required option You may specify a required (mandatory) option using <code>.requiredOption()</code>. The option
<pre>40 \$ collect -n 1 2 3 --letter a b c options: { number: ['1', '2', '3'], [] } program.parse(); console.log('Remaining arguments: ', program.opts()); console.log('Options: ', program.parse()); .option('-l, --letter [letters...]', 'specify letters');</pre>	<pre>39 program .option('-n, --number <numbers...>', 'specify numbers') Example file: options-variadic.js further values are read. the same argument as the option then no processing entirely. If a value is specified in The special argument -- stops option until the first argument starting with a dash. be an array. The extra arguments are read arguments, and the parsed option value will</pre>	<pre>38 You may make an option variadic by appending ... to the value placeholder when declaring the option. On the command line you can then specify multiple option- Variadic option \$ pizza error: required option '-c, --cheese <type>' not specified program.parse();</pre>	<pre>37 program .requiredOption('-c, --cheese <type>', 'pizza must have cheese'); Example file: options-required.js custom processing. description, and optional default value or .option() in format, taking flags and from a default value (say from environment). specified on the command line, or perhaps must have a value after parsing, usually</pre>
<pre>letter: ['a', 'b', 'c'] } Remaining arguments: [] \$ collect --letter=A -n80 operand Options: { number: ['80'], letter: ['A'] } Remaining arguments: ['operand'] \$ collect --letter -n 1 -n 2 3 --operand Options: { number: ['1', '2', '3'], letter: true } Remaining arguments: ['operand']</pre>	For information about possible ambiguous cases, see options taking varying arguments. Version option The optional version method adds handling for displaying the command version. The default option flags are -V and --version, and when present the command prints the version number and exits. <pre>program.version('0.0.1');</pre>	<pre>\$./examples/pizza -V 0.0.1 You may change the flags and description by passing additional parameters to the version method, using the same syntax for flags as the option method. program.version('0.0.1', '-v, --vers', 'output the current version');</pre>	More configuration You can add most options using the <code>.option()</code> method, but there are some additional features available by constructing an Option explicitly for less common cases. Example files: options-extra.js, options-env.js, options-conflicts.js, options-implies.js <pre>program .addOption(new Option('-s, --secret').hideHelp())</pre>
<pre>41 \$ extra --drink huge error: option '-d, --drink <size>' argument 'huge' is invalid. Allowed choices are small, medium, large. \$ extra --drink huge \$ PORT=80 extra --donate --free-drink server --free-drink small drink included free -h, --help display help for command</pre>	<pre>42 Options: -t, --timeout <delay> timeout in seconds (default: one minute) -d, --drink <size> drink cup size (choices: "small", "medium", "large") -p, --port <number> port number (env: PORT) --donate [amount] optional donation in dollars (preset: "20") --disable-server disables the</pre>	<pre>43 .addOption(new Option('--donate [amount]', 'optional donation in dollars').preset('20').argParser((param) => { if (param === 'small') { return 1; } if (param === 'medium') { return 2; } if (param === 'large') { return 3; } return 0; })).choices(['small', 'medium', 'large'])).addOption(new Option('-d, --drink <size>', 'drink size').choices(['small', 'medium', 'large'])).addOption(new Option('-p, --port <number>', 'port').env('PORT')).implies({ drink: true }); Usage: help [options] \$ extra --help</pre>	<pre>44 .addOption(new Option('-d, --drink <size>', 'drink size').choices(['small', 'medium', 'large'])).addOption(new Option('-p, --port <number>', 'port').env('PORT'))</pre>
<pre>48 server --free-drink small drink included free -h, --help display help for command</pre>	<pre>47 Options: -t, --timeout <delay> timeout in seconds (default: one minute) -d, --drink <size> drink cup size (choices: "small", "medium", "large") -p, --port <number> port number (env: PORT) --donate [amount] optional donation in dollars (preset: "20") --disable-server disables the</pre>	<pre>46 .addOption(new Option('--donate [amount]', 'optional donation in dollars').preset('20').argParser((param) => { if (param === 'small') { return 1; } if (param === 'medium') { return 2; } if (param === 'large') { return 3; } return 0; })).addOption(new Option('-d, --drink <size>', 'drink size').choices(['small', 'medium', 'large'])).addOption(new Option('-p, --port <number>', 'port').env('PORT')).implies({ drink: true }); Usage: help [options] \$ extra --help</pre>	<pre>45 .addOption(new Option('-d, --drink <size>', 'drink size').choices(['small', 'medium', 'large'])).addOption(new Option('-p, --port <number>', 'port').env('PORT'))</pre>

Options: { timeout: 60, donate: 20, port: '80', freeDrink: true, drink: 'small' } <pre>\$ extra --disable-server --port 8000 error: option '--disable-server' cannot be used with option '-p, --port <number>'</pre> Specify a required (mandatory) option using the Option method .makeOptionMandatory(). This matches the Command method .requiredOption(). 49	Custom option processing You may specify a function to do custom processing of option-arguments. The callback function receives two parameters, the user specified option-argument and the previous value for the option. It returns the new value for the option. This allows you to coerce the option-argument to the desired type, or accumulate values, or do entirely custom processing. 50	You can optionally specify the default/starting value for the option after the function parameter. Example file: options-custom-processing.js <pre>function myParseInt(value, dummyPrevious) { // parseInt takes a string and a radix const parsedValue = parseInt(value, 10); if (isNaN(parsedValue)) {</pre> 51	<pre> throw new commander.InvalidArgumentError('Not a number.');</pre> <pre> } return parsedValue; }</pre> <pre>function increaseVerbosity(dummyValue, previous) { return previous + 1; }</pre> 52
<pre>float: 100 \$ custom -f 1e2 console.log(options.list); if (options.list !== undefined) console.log(options.collect); if (options.collect.length < 4) {options.verbose}++; console.log(`verbosity: \$   @ < verbose @esob`); if (options.verbose < 4) {options.integer}++; console.log(`integer: \$   undefined`); if (options.integer !== undefined)</pre> 55	<pre>{options.float})); console.log(`float: \$   undefined`); const options = program.opts(); program.parse(); : comma separated list', commaSeparatedList</pre> 55	<pre>.option('-f, --float <number>', 'float argument', parseFloat) .option('-i, --integer <number>', 'integer argument', myParseInt) .option('-v, --verbose', 'verbosity that can be increased', increaseVerbosity, 0) .option('-c, --collect <value>', 'repeatable value', collect, [])</pre> 54	<pre>function collect(value, previous) { return previous.concat([value]); } function commaSeparatedList(value, dummyPrevious) { return value.split(',').join(' '); } program</pre> 53
<pre>\$ custom --integer 2 integer: 2 \$ custom -v -v -v verbose: 3 \$ custom -c a -c b -c c ['a', 'b', 'c'] \$ custom --list x,y,z ['x', 'y', 'z']</pre> Commands You can specify (sub)commands using .command() or .addCommand(). There are two 57	ways these can be implemented: using an action handler attached to the command, or as a stand-alone executable file (described in more detail later). The subcommands may be nested (example). In the first parameter to .command() you specify the command name. You may append the command-arguments after the command name, or specify them separately using .argument(). The arguments may be <required> or [optional], and the last argument may also be variadic.... 58	You can use .addCommand() to add an already configured subcommand to the program. For example: <pre>// Command implemented using action handler (description is supplied separately to `.command`) // Returns new command for configuring. program</pre> 59	<pre>.command('clone <source> [destination]') .description('clone a repository into a newly created directory') .action((source, destination) => { console.log('clone command called'); });</pre> 60
For subcommands, you can specify the argument syntax in the call to .command() (as shown above). This is the only method usable for subcommands implemented using a stand-alone executable, but for other subcommands you can instead use the following method. To configure a command, you can use .argument() to specify each expected command-argument. You supply the 64	command from the generated help output. Specifying isDefault: true will run the subcommand if no other subcommand is specified (example). You can add alternative names for a command with .aliases(). (example) For safety, .addCommand() does not automatically copy the inherited settings from the parent command. There is a helper routine .copyInheritedSettings() for copying the settings when they are wanted. 63	<pre>.command('stop [service]', 'stop named service, or all if no name supplied'); // Command prepared separately. // Returns `this` for adding more commands. program .addCommand(build.makeBuildCommand()); Configuration options can be passed with the call to .command() and .addCommand(). Specifying hidden: true will remove the</pre> 62	<pre>// Command implemented using stand- alone executable file, indicated by adding description as second parameter to `.command`. // Returns `this` for adding more commands. program .command('start <service>', 'start named service')</pre> 61

argument name and an optional description. The argument may be <required> or [optional]. You can specify a default value for an optional command-argument. Example file: argument.js <pre> program .version('0.1.0') .argument('<username>', 'user to login') .argument('[password]', 'password for user, if required', 'no password given') </pre>	<pre> .action((username, password) => { console.log('username:', username); console.log('password:', password); }); </pre> The last argument of a command can be variadic, and only the last argument. To make an argument variadic you append ... to the argument name. A variadic argument is passed to the action handler as an array. For example:	<pre> program .version('0.1.0') .command('rmdir') .argument('<dirs...>') .action(function (dirs) { dirs.forEach((dir) => { console.log('rmdir %s', dir); }); }); </pre> There is a convenience method to add multiple arguments at once, but without descriptions:	<pre> program .arguments('<username> <password>'); </pre> More configuration There are some additional features available by constructing an Argument explicitly for less common cases. Example file: arguments-extra.js
The action handler gets passed a parameter for each command-argument you declared, and two additional parameters which are the parsed options and the command object itself. Example file: thank.js Action handler <pre> ; {} = \${first} + \${second} console.log(`\${first} + \${second}`); </pre>	<pre> program .command('add') .argument('<first>', 'integer') .argument('[second]', 'integer') .action((first, second) => { argument('myParseInt, 1000') }); </pre> processing.js Example file: arguments-custom-processing.js You can optionally specify the default/starting value for the argument after the function parameter.	You may specify a function to do custom processing of command-arguments (like for option-arguments). The callback function receives two parameters, the user specified command-argument and the previous value for the argument. It returns the new value for the argument. The processed argument values are passed to the action handler, and saved as .processedArgs. 70	<pre> program .addArgument(new commander.Argument('drink-size', 'drink cup size', ['small', 'medium', 'large'])) .addArgument(new commander.Argument('timeout', 'timeout in seconds', default(60, 'one minute'))); </pre> 69
<pre> program .argument('<name>') .option('-t, --title <honorific>', 'title to use before name') .option('-d, --debug', 'display some debugging') .action((name, options, command) => { if (options.debug) { console.error('Called %s with options %o', command.name(), options); } }); </pre> 73	<pre> } const title = options.title ? `\${options.title}` : ''; console.log(`Thank-you \${title}\${name}`); }); </pre> If you prefer, you can work with the command directly and skip declaring the parameters for the action handler. The this keyword is set to the running command and can be used from a function expression (but not from an arrow function).	Example file: action-this.js <pre> program .command('serve') .argument('<script>') .option('-p, --port <number>', 'port number', 80) .action(function() { console.error('Run script %s on port %s', this.args[0], this.opts().port); }); </pre> 75	You may supply an async action handler, in which case you call .parseAsync rather than .parse. <pre> async function run() { /* code goes here */ } async function main() { program .command('run') .action(run); await program.parseAsync(process.argv); } </pre> 76
<pre> .command('install [name]', 'install one or more packages') .command('search [query]', 'search with optional query') .command('update', 'update installed packages', { executableFile: 'updatesSubCommand' }) .command('list', 'list packages installed', { isDefault: true }); </pre> 79	<pre> .version('0.1.0') .name('pm') </pre> Example file: pm declare them at the top-level. (sub)command in the executable, and don't You handle the options for an executable subcommands with .executableDir(). may specify a custom search directory for executableFile configuration option. You custom name (and path) with the extensions, like .js. You may specify a	The search includes trying common file install or pm-search in the example below. combination command-subcommand, like pm-entry script for a file with the name will search the files in the directory of the executables for subcommands. Commander that you're going to use stand-alone description argument, this tells Commander When .command() is invoked with a (sub)commands Stand-alone executable 78	A command's options and arguments on the command line are validated when the command is used. Any unknown options or missing arguments will be reported as an error. You can suppress the unknown option checks with .allowUnknownOption(). By default it is not an error to pass more arguments than declared, but you can make this an error with .allowExcessArgs(false). 77

<pre>program.parse(process.argv);</pre> If the program is designed to be installed globally, make sure the executables have proper modes, like 755. Life cycle hooks You can add callback hooks to a command for life cycle events. Example file: hook.js 81	<pre>program .option('-t, --trace', 'display trace statements for commands') .hook('preAction', (thisCommand, actionCommand) => { if (thisCommand.opts().trace) { console.log(`About to call         action handler for         subcommand: \$         {actionCommand.name()}`); } });</pre> 82	<pre>console.log('arguments: %0', actionCommand.args); console.log('options: %o', actionCommand.opts()); }</pre> The callback hook can be async, in which case you call <code>.parseAsync</code> rather than <code>.parse</code>. You can add multiple hooks per event. The supported events are: 83	<table><tr><th>event name</th><th>when hook called</th><th>callback parameters</th></tr><tr><td>preAction, postAction</td><td>before/after action handler for this command and its nested subcommands</td><td>(thisCommand, actionCommand)</td></tr><tr><td>preSubcommand</td><td>before parsing direct subcommand</td><td>(thisCommand, subcommand)</td></tr></table> 84	event name	when hook called	callback parameters	preAction, postAction	before/after action handler for this command and its nested subcommands	(thisCommand, actionCommand)	preSubcommand	before parsing direct subcommand	(thisCommand, subcommand)
event name	when hook called	callback parameters										
preAction, postAction	before/after action handler for this command and its nested subcommands	(thisCommand, actionCommand)										
preSubcommand	before parsing direct subcommand	(thisCommand, subcommand)										
88 Custom help Long descriptions are wrapped to fit the available width. (However, a description that includes a line-break followed by whitespace is assumed to be pre-formatted and not wrapped.) You can add extra text to be displayed along with the built-in help. Example file: custom-help	<pre>shell help shell help spawn shell --help shell help implicit help: effectively the same if the shell program has further help for the subcommand. These are alone, or with a subcommand name to show command has subcommands. It can be used by default if your A help command is added</pre> 87	<pre>An application for pizza ordering Options: -p, --peppers Add peppers -c, --cheese <type> Add the specified type of cheese (default: "marble") -C, --no-cheese You do not want any cheese -h, --help display help for command</pre> 86	For an overview of the life cycle events see parsing life cycle and hooks. Automated help The help information is auto-generated based on the information commander already knows about your program. The default help option is <code>-h, --help</code>. Example file: pizza <code>\$ node ./examples/pizza --help</code> Usage: pizza [options] 85									
<pre>program .option('-f, --foo', 'enable some foo'); program.addHelpText('after', `   Example call:     \$ custom-help --help`); Yields the following help output: Usage: custom-help [options]</pre> 89	Options: <pre>-f, --foo enable some foo -h, --help display help for command</pre> Example call: <pre>\$ custom-help --help</pre> The positions in order displayed are: - beforeAll: add to the program for a global banner or header - before: display extra information before built-in help 90	- after: display extra information after built-in help - afterAll: add to the program for a global footer (epilog) The positions “beforeAll” and “afterAll” apply to the command and all its subcommands. The second parameter can be a string, or a function returning a string. The function is 91	passed a context object for your convenience. The properties are: - error: a boolean for whether the help is being displayed due to a usage error - command: the Command which is displaying the help Display help after errors The default behaviour for usage errors is to just display a short error message. You can 92									
96 name The command name appears in the help, and is also used for locating stand-alone executable subcommands. You may specify the program name using <code>.name()</code> or in the Command constructor. For the program, Commander will fallback to using the script name from the full arguments passed into <code>.parse()</code>. However, the script name varies depending on how your program	<pre>.help(): display help information and exit immediately. You can optionally pass { error: true } to display on stderr and exit with an error status. .outputHelp(): output help information without exiting. You can optionally pass { error: true } to display on stderr. .helpInformation(): get the built-in command help information as a string for processing or displaying yourself.</pre> 95	<pre>program.showSuggestionAfterError(false); \$ pizza --help error: unknown option '--help' (program help for additional information): \$ pizza --unknown error: unknown option '--unknown' (add --help for additional information)</pre> The default behaviour is to suggest correct spelling after an error for an unknown command or option. You can disable this. 94	a custom help message after an error. change the behaviour to show the full help or <pre>program.showHelpAfterError(): // or program.showHelpAfterError('(add -- help for additional information)'); \$ pizza --unknown error: unknown option '--unknown' (add --help for additional information)</pre> 93									

is launched so you may wish to specify it explicitly. <pre>program.name('pizza'); const pm = new Command('pm');</pre> Subcommands get a name when specified using <code>.command()</code>. If you create the subcommand yourself to use with <code>.addCommand()</code>, then set the name using <code>.name()</code> or in the <code>Command</code> constructor.	.usage This allows you to customise the usage description in the first line of the help. Given: <pre>program .name("my-command") .usage("[global options] command")</pre> The help will start with: <pre>Usage: my-command [global options] command</pre>	.description and .summary The description appears in the help for the command. You can optionally supply a shorter summary to use when listed as a subcommand of the program. <pre>program .command("duplicate") .summary("make a copy") .description('Make a copy of the current project.</pre>	This may require additional disk space. <pre>`);</pre> .helpOption(flags, description) By default every command has a help option. You may change the default help flags and description. Pass false to disable the built-in help option.
101 <pre>sortSubcommands: true, program.configureHelp({ Example file: configure-help.js used. look at <code>.formatHelp()</code> to see how they are formatting the items in the lists, with each item having a <i>term</i> and <i>description</i>. Take a look at <code>.formatHelp()</code> to see how they are used. lists of arguments, options, and subcommands. There are methods for formatting the items in the lists, with each item having a <i>term</i> and <i>description</i>. Take a look at <code>.formatHelp()</code> to see how they are used. You can override any method on the Help class. There are methods getting the visible lists of arguments, options, and subcommands. There are methods for formatting the items in the lists, with each item having a <i>term</i> and <i>description</i>. Take a look at <code>.formatHelp()</code> to see how they are used.</pre>	103 <pre>using <code>.configureHelp()</code>, or by subclassing using <code>.createHelp()</code> if you prefer. The data properties are: • <code>helpWidth</code>: specify the wrap width, useful for unit tests • <code>sortSubcommands</code>: sort the subcommands alphabetically • <code>sortOptions</code>: sort the options alphabetically • <code>showGlobalOptions</code>: show a section with the global options from the parent command(s)</pre>	102 You can both turn on and customise the help command by supplying the name and description: <pre>program.addHelpCommand('assist [command]', 'show assistance');</pre> The built-in help is formatted using the Help class. You can configure the Help behaviour by modifying data properties and methods More configuration	101 <pre>program .helpOption('-e, -- -HELP', 'read more information');</pre> A help command is added by default if your command has subcommands. You can explicitly turn on or off the implicit help command with <code>.addHelpCommand()</code> and <code>.addHelpCommand(false)</code>.
105 <pre>subcommandTerm: (cmd) => cmd.name() // Just show the name, instead of short usage. });</pre> Custom event listeners You can execute custom actions by listening to command and option events.	106 <pre>program.on('option:verbose', function () { process.env.VERBOSE = this.opts().verbose; });</pre>	107 Bits and pieces .parse() and .parseAsync() The first argument to <code>.parse</code> is the array of strings to parse. You may omit the parameter to implicitly use <code>process.argv</code>.	108 If the arguments follow different conventions than node you can pass a from option in the second parameter: <code>'node'</code>: default, <code>argv[0]</code> is the application and <code>argv[1]</code> is the script being run, with user parameters after that <code>'electron'</code>: <code>argv[1]</code> varies depending on whether the electron application is packaged <code>'user'</code>: all of the arguments from the user For example:
112 <pre>With pass through options, the --port=80 is a program option in the first line and passed through as a command-argument in the second line: program arg --port=80 arg program arg --port=80</pre> By default the option processing shows an error for an unknown option. To have an unknown option treated as an ordinary command-argument and continue looking for	111 <pre>By default options are recognised before and after command-arguments. To only process options that come before the command-arguments, use <code>passThroughOptions()</code>. This lets you pass the arguments and following options through to another program without needing to use -- and the option processing. To use pass through options in a subcommand, the program needs to enable positional options. Example file: pass-through-options.js</pre>	110 <pre>By default program options are recognised before and after subcommands. To only look for program options before subcommands, use <code>enablePositionalOptions()</code>. This lets you use an option for a different purpose in subcommands. Example file: positional-options.js option in the first line and a subcommand option in the second line: program -b subcommand program -b subcommand</pre>	109 Parsing Configuration <pre>program.parse(argv); // explicit, node conventions and auto-detect electron program.parse(['-f', 'filename'], { from: 'user' });</pre> If the default parsing does not suit your needs, there are some behaviours to support other usage patterns.

options, use <code>.allowUnknownOption()</code>. This lets you mix known and unknown options. By default the argument processing does not display an error for more command-arguments than expected. To display an error for excess arguments, use <code>.allowExcessArguments(false)</code>. Legacy options as properties Before Commander 7, the option values were stored as properties on the command.	This was convenient to code but the downside was possible clashes with existing properties of <code>Command</code>. You can revert to the old behaviour to run unmodified legacy code by using <code>.storeOptionsAsProperties()</code>. <pre>program .storeOptionsAsProperties() .option('-d, --debug') .action((commandAndOptions) => { if (commandAndOptions.debug) { console.error(`Called \$         {commandAndOptions.name()}`); } });</pre>	<pre>} });</pre> TypeScript extra-typings: There is an optional project to infer extra type information from the option and argument definitions. This adds strong typing to the options returned by <code>.opts()</code> and the parameters to <code>.action()</code>. See commander-js/extra-typings for more.	<pre>import { Command } from '@commander- js/extra-typings'; ts-node: If you use ts-node and stand- alone executable subcommands written as .ts files, you need to call your program through node to get the subcommands called correctly. e.g. node -r ts-node/register pm.ts</pre>
113	114	115	116
120 executable subcommands you need to set the If you are using VSCode to debug subcommand. incremented by 1 for the spawned node --inspect et al, the inspector port is debugging executable subcommands using If you are using the node inspector for a separate child process. An executable subcommand is launched as subcommands Debugging stand-alone executable	119 spawning subcommand process. --harmony option will be preserved when node --harmony examples/pm publish. The command, like • Use the --harmony option when call the does not support this pattern.) the subcommands scripts. (Note Windows • Use #!usr/bin/env node --harmony in ways: You can enable --harmony option in two Node options such as --harmony	118 command-class.js. subcommand (example file custom- may override it to customise the new subcommands using .command(), and you gets used internally when creating	117 command rather than a subcommand. This Command object, and creates a new createCommand is also a method of the <pre>const program = createCommand(); require('commander'); const { createCommand } =</pre> instead of using new, like: This factory function creates a new command. It is exported and may be used createCommand()
"autoAttachChildProcesses": true flag in your launch.json configuration. Display error This routine is available to invoke the Commander error handling for your own error conditions. (See also the next section about exit handling.) As well as the error message, you can optionally specify the <code>exitCode</code> (used with	process.exit) and code (used with <code>CommanderError</code>). <pre>program.error('Password must be longer than four characters'); program.error('Custom processing has failed', { exitCode: 2, code: 'my.custom.error' });</pre>	Override exit and output handling By default Commander calls <code>process.exit</code> when it detects errors, or after displaying the help or version. You can override this behaviour and optionally supply a callback. The default override throws a <code>CommanderError</code>. The override callback is passed a <code>CommanderError</code> with properties <code>exitCode</code> number, <code>code</code> string, and <code>message</code>. The default override behaviour is to throw the	error, except for async handling of executable subcommand completion which carries on. The normal display of error messages or version or help is not affected by the override which is called after the display. <pre>program.exitOverride(); try { program.parse(process.argv); } catch (err) { // custom processing... }</pre>
121	122	123	124
128 Available as part of the Tidelfit Subscription Commander for enterprise The main forum for free and community support is the project issues on GitHub. Commander) versions of Node.js, use an older version of Node.js, and requires at least v14. (For older supported on Long Term Support versions of The current version of Commander is fully Support	127 There is more information available about: • deprecated features still supported for backwards compatibility • options taking varying arguments • parsing life cycle and hooks Additional documentation <pre>outputError: (str, write) => write(errorColor(str)) }) ;</pre>	126 <pre>program .configureOutput({ // Visibly override write routines as example! writeOut: (str) => process.stdout.write(`[OUT] \$         {str}`), writeErr: (str) => process.stdout.write(`[ERR] \$         {str}`). // Highlight errors in color.</pre>	125 By default Commander is configured for a command-line application and writes to stdout and stderr. You can modify this behaviour for custom applications. In addition, you can modify the display of error messages. Example file: configure-output.js <pre>function errorColor(str) { // Add ANSI escape codes to display text in red. return `\\x1b[31m\${str}\\x1b[0m`; }</pre>

The maintainers of Commander and thousands of other packages are working with Tidelift to deliver commercial support and maintenance for the open source dependencies you use to build your applications. Save time, reduce risk, and improve code health, while paying the maintainers of the exact dependencies you use. Learn more.			
129	130	131	132
136	135	134	133
137	138	139	140
144	143	142	141